STOCK MARKET ANALYZER

A PROJECT REPORT

Submitted by

Student Name	Student Code	Registration Number	Roll Number
SK ARIF HOSSAIN	BWU/MCA/26/046	26102000163	26102410041
PUSKAR MONDAL	BWU/MCA/26/047	26102000164	26102410042
ABESH CHANDRA SARKAR	BWU/MCA/26/048	26102000165	26102410043
SIPRA KUNDU	BWU/MCA/26/049	26102000166	26102410044
MIHIR ROY	BWU/MCA/26/056	26102000173	26102410051

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

IN

COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to the Management and Faculty of Brainware University for providing us with the opportunity, guidance, and resources required to successfully complete this project. Their continuous support and encouragement have played an important role in our learning and development throughout the project.

We sincerely thank our project supervisor for providing valuable guidance, suggestions, and feedback during the development of the Stock Market Analyzer. Their support helped us understand the practical aspects of web development, API integration, data processing, visualization, and user interface design.

We are also deeply thankful to the Head of the Department of Computational Sciences for providing a supportive academic environment and the necessary facilities required to complete this project successfully.

Our sincere appreciation goes to all the faculty members of the Department of Computational Sciences for sharing their knowledge, advice, and encouragement. Their teachings helped us strengthen our technical understanding and apply theoretical concepts in a practical project.

Finally, we express our profound gratitude to our families and friends for their constant support, motivation, and encouragement throughout the development of this project. Their valuable support inspired us to remain committed and complete the project successfully.

SIGNATURE:-

SK ARIF HOSSAIN

SIPRA KUNDU

MIHIR ROY

PUSKAR MONDAL

ABESH CHANDRA SARKAR

ABSTRACT

The **Stock Market Analyzer** is a web-based application developed to provide users with an interactive platform for searching and analyzing stock market information. The main objective of this project is to demonstrate the practical implementation of modern web technologies while providing an easy-to-use interface for viewing stock prices, daily market information, historical price trends, and watchlist data.

The application is developed using **HTML5, CSS3, and JavaScript**, with the **Alpha Vantage API** used to retrieve stock market data. The application processes the received data and presents important information such as the latest closing price, price change, percentage change, opening price, highest price, lowest price, previous closing price, and trading volume.

The system provides a stock search facility through which users can enter a stock symbol such as AAPL, MSFT, GOOGL, TSLA, or AMZN. After receiving the requested data, JavaScript processes the API response and dynamically updates the user interface without requiring a complete page reload.

The project also includes a graphical representation of historical closing prices using **Chart.js.** The application displays the closing prices from the latest 30 available trading days in a responsive line chart, allowing users to understand the recent movement and trend of a selected stock more easily.

A **Watchlist** feature is also implemented using browser LocalStorage. Users can add stocks to their watchlist and access them later from the same browser. This provides a simple way to keep frequently viewed stocks available for quick analysis.

Overall, the Stock Market Analyzer demonstrates important concepts including API integration, asynchronous JavaScript, JSON data processing, DOM manipulation, data visualization, LocalStorage, responsive web design, and interactive user interfaces.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	Acknowledgement	i
	Abstract	ii
	Table of Contents	iii
1	Introduction	1
2	Literature Survey	2
3	System Analysis and Design	3-7
4	Implementation	8
5	Output Screenshot	9-10
6	Source Code	11-13
7	Testing and Debugging	14-15
8	Conclusion and Future Enhancement	16-17
9	References	18

1. INTRODUCTION

In today's digital world, financial information is increasingly accessed through online platforms. Stock market data contains a large amount of information such as stock prices, daily price changes, trading volume, opening and closing prices, and historical trends. Presenting this information in a simple and understandable way can help users quickly analyze the recent performance of a stock.

Traditional methods of obtaining stock information may require users to visit different financial websites or applications and navigate through large amounts of information. This can make it difficult for beginners to quickly identify the important information they need.

The **Stock Market Analyzer** is a web-based application developed to provide a simple and interactive solution for viewing stock market information. The application allows users to search for a stock using its symbol and view important market data in an organized dashboard.

The system uses the **Alpha Vantage API** to retrieve stock market data. The received information is processed using JavaScript and displayed dynamically on the webpage. Important values such as the current/latest closing price, opening price, highest price, lowest price, previous closing price, trading volume, price change, and percentage change are presented through separate information cards.

The application also provides graphical visualization of historical stock prices. Using **Chart.js**, the closing prices from the latest 30 available trading days are represented through a line chart. This visualization helps users observe price movements and identify recent trends more easily.

A watchlist feature has also been included in the system. Users can add frequently searched stocks to their watchlist, which is stored in the browser's LocalStorage. Saved stocks can later be selected to perform another analysis without manually entering their symbols again.

The user interface has been designed using HTML5 and CSS3 with a modern dark financial-dashboard appearance. Gradients, cards, responsive layouts, animations, hover effects, and graphical components are used to create an attractive and user-friendly experience.

The project also demonstrates practical concepts of asynchronous programming, API communication, JSON data processing, DOM manipulation, browser storage, responsive design, and data visualization.

In conclusion, the Stock Market Analyzer provides a compact and interactive platform for exploring stock market information while demonstrating the practical application of modern frontend web development technologies.

2. LITERATURE SURVEY

Stock market analysis applications have become increasingly common with the growth of online financial platforms and financial data APIs. Earlier websites generally displayed market information through static HTML pages or server-generated pages. Users often had to refresh pages or navigate between different sections to obtain updated information.

Modern web technologies have improved this process by allowing applications to retrieve data asynchronously and update only the required parts of the webpage. JavaScript, Fetch API, and browser-based APIs make it possible to communicate with external data services without requiring a complete page reload.

Financial APIs such as Alpha Vantage provide developers with access to structured market data. Such APIs allow applications to retrieve stock-related information and process the returned JSON data for presentation to users.

Data visualization is another important part of financial applications. Numerical stock information can be difficult to understand when presented only as tables or individual values. Graphical tools such as Chart.js allow historical values to be represented through line charts, making trends and changes easier to understand.

Modern dashboard design also focuses on presenting large amounts of information in a compact and organized manner. Cards, charts, status indicators, search controls, and responsive layouts are commonly used to improve usability.

The Stock Market Analyzer applies these concepts by combining API-based stock data retrieval, JavaScript data processing, graphical visualization, responsive interface design, and browser-based watchlist storage.

The project therefore provides practical experience in developing an interactive financial information application using frontend web technologies and external API integration.

3. SYSTEM ANALYSIS AND DESIGN

The System Analysis and Design phase is an important stage in the development of the Stock Market Analyzer. It focuses on understanding the requirements of the application, identifying the problems of traditional approaches, defining the proposed solution, and designing the overall structure of the system.

3.1 System Analysis

System analysis focuses on identifying the requirements for accessing and presenting stock market information in a simple and interactive manner.

Existing System

Users commonly obtain stock information through financial websites, mobile applications, or other online platforms. These platforms may provide extensive information, but beginners may find it difficult to identify the specific data they need quickly.

Problems in the Existing Approach

- Stock information may be distributed across multiple sections.
- Large amounts of financial information can make quick analysis difficult.
- Beginners may find complex financial dashboards difficult to understand.
- Historical price trends may not always be presented in a simple visual format.
- Frequently searched stocks may require repeated manual searches.

Proposed System

The proposed **Stock Market Analyzer** is a web-based application that provides a simple interface for searching and analyzing stock market information.

Users can enter a stock symbol and retrieve market information through the Alpha Vantage API. The application processes the received data and presents it through organized information cards and graphical charts.

Advantages of the Proposed System

- Simple and user-friendly stock search interface.
- Dynamic retrieval of stock market information.

- Displays important stock price statistics in an organized manner.
- Provides historical closing-price visualization.
- Displays the latest 30 available trading-day prices through a chart.
- Allows users to maintain a personal watchlist.
- Uses LocalStorage to preserve watchlist information in the browser.
- Responsive interface suitable for desktop and smaller screens.

3.2 System Design

System design describes the structure of the Stock Market Analyzer and explains how its different components work together.

Modules of the System

1. Stock Search Module
2. Stock Data Analysis Module
3. Historical Chart Module
4. Watchlist Module
5. Responsive User Interface Module

Stock Search Module

The Stock Search Module allows users to enter a stock symbol and search for its market information. The entered symbol is converted to uppercase and validated before the API request is performed.

Popular stock shortcuts such as AAPL, MSFT, GOOGL, TSLA, and AMZN are also provided for quick searching.

Stock Data Analysis Module

After receiving the API response, JavaScript processes the daily time-series data. The system identifies the latest available trading-day record and extracts important information including:

- Latest closing price
- Opening price
- Highest price
- Lowest price

- Previous closing price
- Trading volume
- Price change
- Percentage change

Historical Chart Module

The Historical Chart Module uses Chart.js to display the closing prices from the latest 30 available trading days.

The data is converted into chart labels and values and displayed through a responsive line chart. The previous chart instance is destroyed before a new chart is generated to prevent multiple charts from overlapping.

Watchlist Module

The Watchlist Module allows users to save frequently searched stock symbols. The watchlist is stored in the browser using LocalStorage.

Users can add a stock to the watchlist, remove it, or select a saved stock to perform another search.

Responsive User Interface Module

The interface is designed using responsive CSS so that the application can adapt to different screen sizes.

The layout includes:

- Navigation/header section
- Hero section
- Search section
- Stock information cards
- Chart section
- Market flow section
- Watchlist section
- About/project section
- Footer

3.3 System Architecture

The Stock Market Analyzer follows a client-side web application architecture**.**

The basic flow of the system is:

User → Stock Search → JavaScript → Alpha Vantage API → JSON Data → Data Processing → Dashboard → Chart / Watchlist

The user enters a stock symbol through the interface. JavaScript sends a request to the Alpha Vantage API using the Fetch API. The API returns structured JSON data containing daily stock information.

The application processes the response, extracts the required values, calculates the price change and percentage change, and updates the webpage dynamically.

The historical closing prices are passed to Chart.js for graphical visualization, while watchlist information is stored using LocalStorage.

Technologies Used:

CATEGORY	TECHNOLOGY
FRONTEND	HTML5
STYLING	CSS3
PROGRAMMING	JavaScript
API COMMUNICATION	Fetch API
MARKET DATA	Alpha Vantage API
DATA VISUALIZATION	Chart.js
BROWSER STORAGE	LocalStorage
DEVELOPMENT	Visual Studio Code / Web Browser

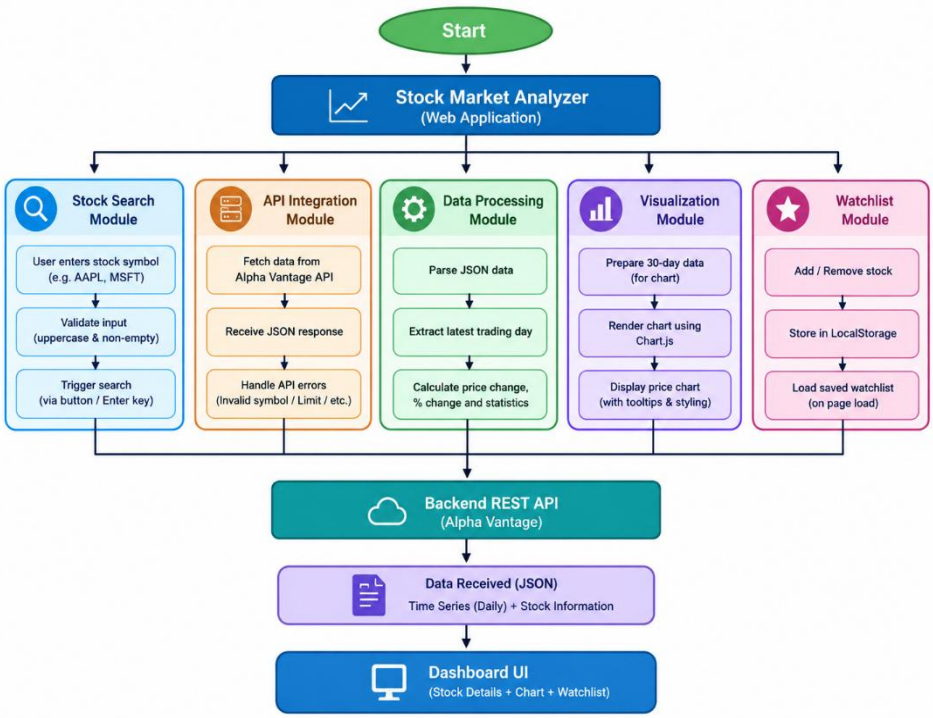


Fig. no. 3.1 – Module and data flow diagram

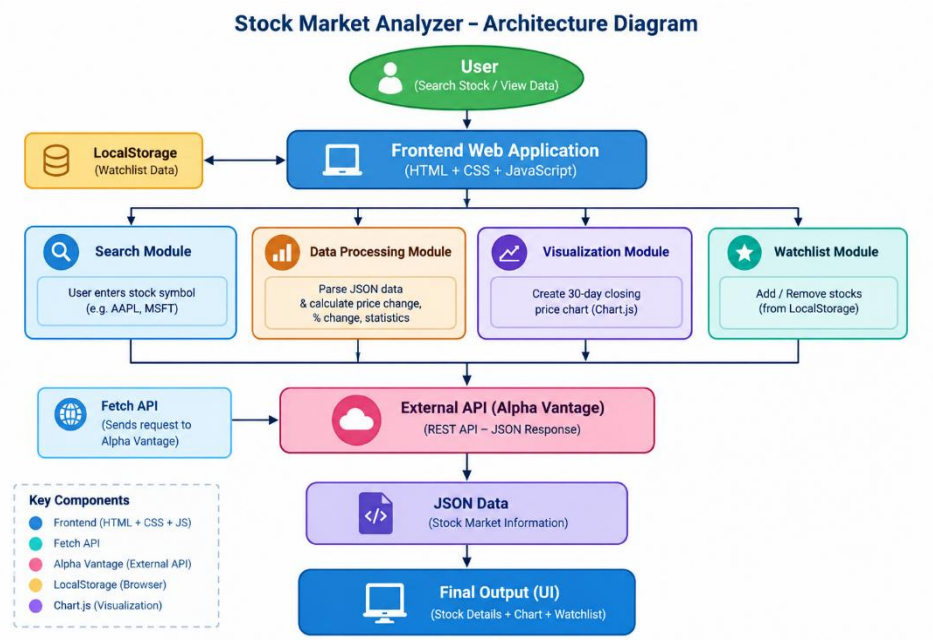


Fig. no. 3.2 – Stock Market Analyzer Architecture Diagram

Overall, the system design ensures that the Doctor Booking System is modular, responsive, and capable of handling real-time appointment data effectively, while giving hands-on experience with the core building blocks of React JS.

4. IMPLEMENTATION

The implementation phase involves developing the Stock Market Analyzer according to the system design.

The application is developed using HTML5, CSS3, and vanilla JavaScript. The HTML file defines the structure of the application, CSS provides the visual design and responsive layout, and JavaScript controls the application logic and dynamic interaction.

The application interface contains a navigation header, hero section, search area, stock analysis section, chart section, market flow section, watchlist section, project information section, and footer.

The stock search functionality is implemented using JavaScript event handling. Users can search for a stock using the search button or by pressing the Enter key after entering a stock symbol.

The entered stock symbol is normalized by removing unnecessary spaces and converting it to uppercase. Empty input is rejected before an API request is made.

The Fetch API is used to communicate with the Alpha Vantage service. After receiving the response, the application checks for HTTP errors and API-related error messages before processing the returned data.

The daily time-series data is extracted from the API response. The latest available record is used to obtain the current/latest closing price and other daily statistics.

The application also obtains the previous trading day's closing price and uses it to calculate the price change and percentage change.

The calculated information is dynamically inserted into the appropriate HTML elements so that the user can immediately see the selected stock's information.

For graphical visualization, the latest 30 available trading-day closing prices are extracted from the API response and passed to Chart.js. The chart is configured to be responsive and includes tooltips, formatted price values, and date labels.

The Watchlist functionality uses LocalStorage to save stock symbols in the browser. When a user adds a stock, the symbol is stored locally. When the application loads, the stored watchlist can be retrieved and displayed.

The implementation therefore combines frontend structure, styling, JavaScript programming, external API communication, data processing, data visualization, and browser storage into a single interactive application.

5. OUTPUT SCREENSHOT

The screenshots below are placeholders showing where each page of the running application should be inserted; replace each placeholder image with the corresponding screenshot captured from the deployed React application.

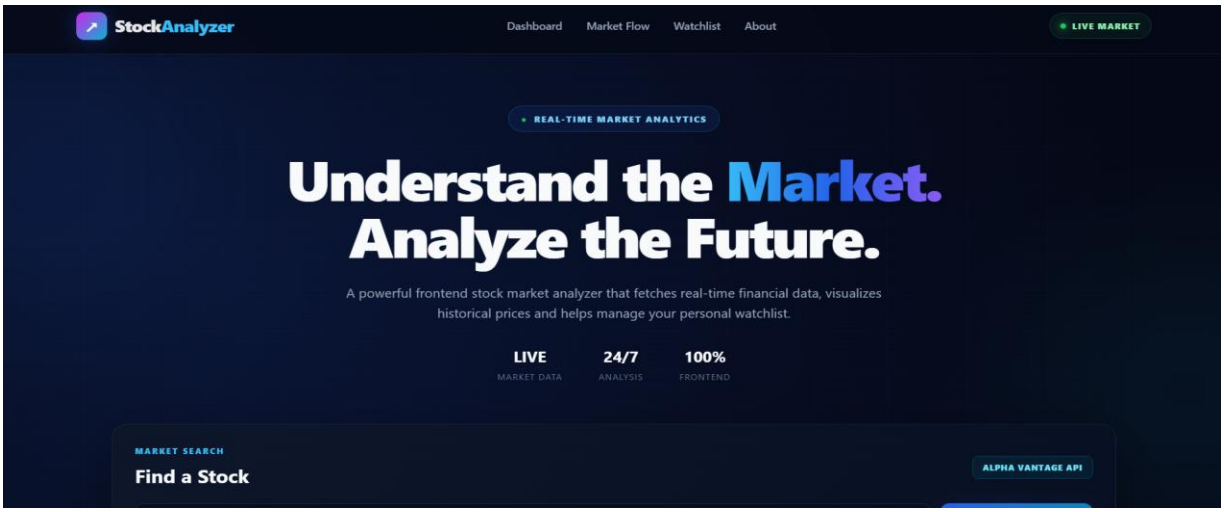


Fig 5.1 - Home Page

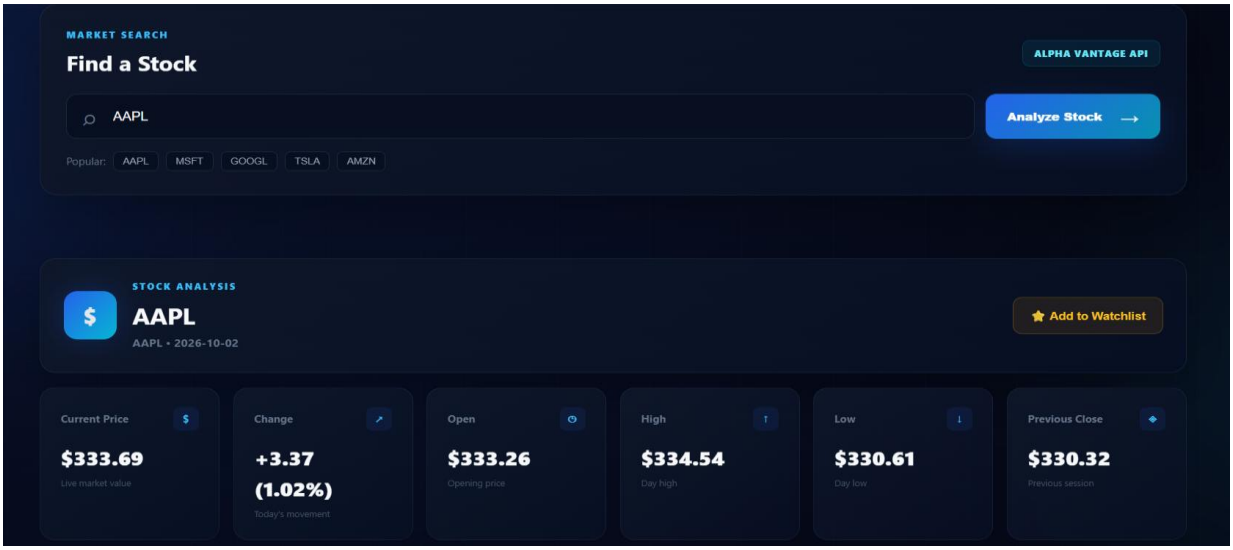


Fig 5.2 - Stock Search Interface

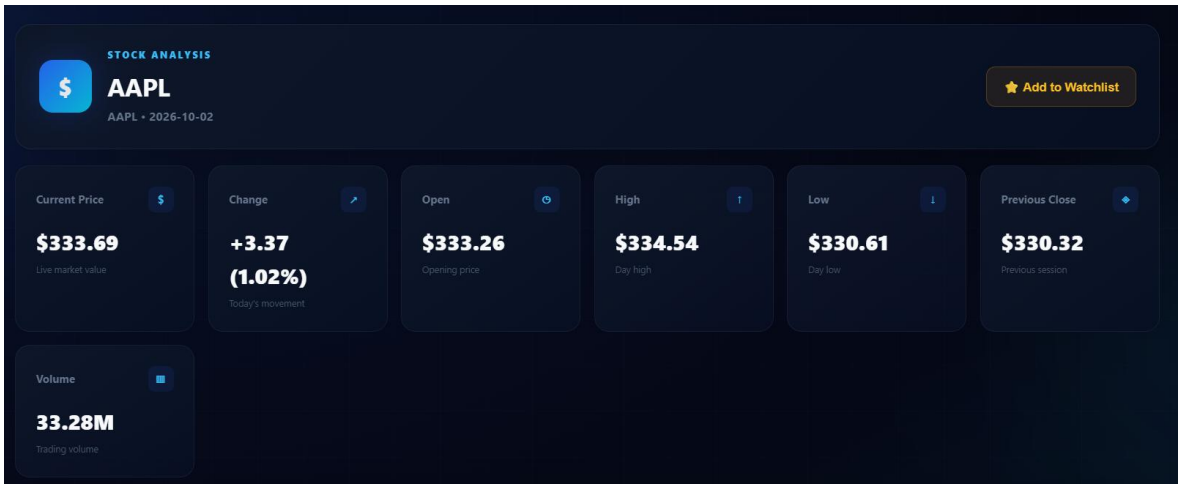


Fig 5.3 - Stock Analysis Details

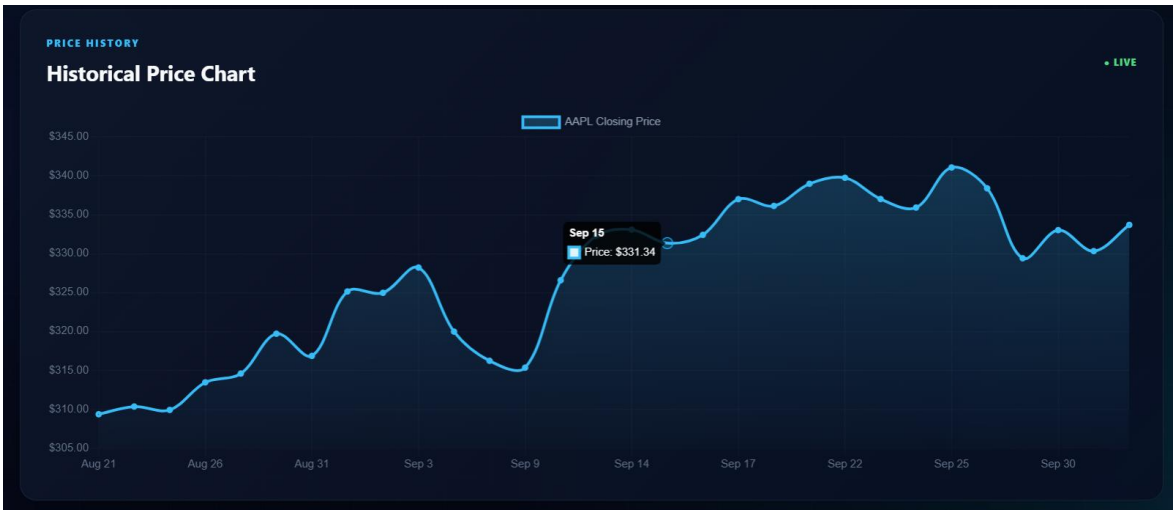


Fig 5.4 - Historical Price Chart

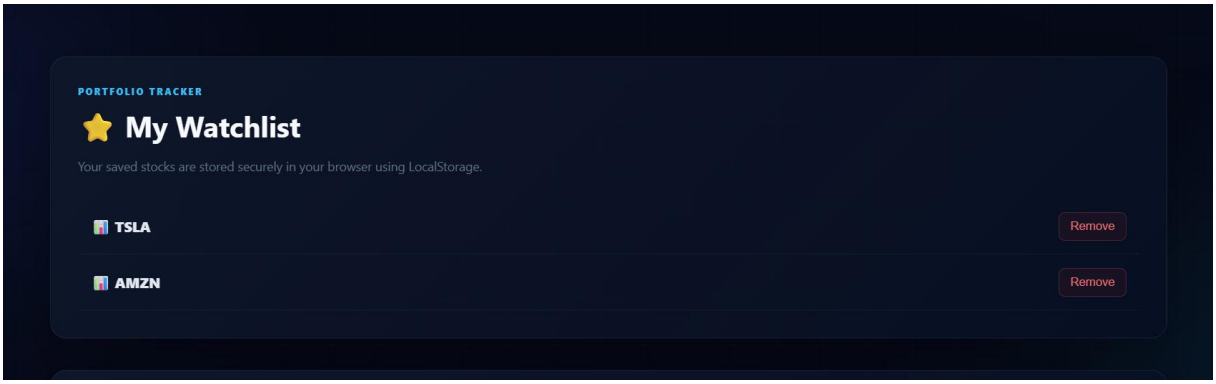


Fig 5.5 - Stock Watchlist

6. SOURCE CODE

6.1 index.html – Main Structure

```
<!-- Search Section -->

<section class="search-section">

  <div class="search-container">

    <input

      type="text"

      id="stockInput"

      placeholder="Enter stock symbol (e.g., AAPL)"

      autocomplete="off"

    >

    <button id="searchBtn">

      Search Stock

    </button>

  </div>

  <div class="popular-stocks">

    <span>Popular:</span>

    <button class="stock-btn" data-symbol="AAPL">AAPL</button>

    <button class="stock-btn" data-symbol="MSFT">MSFT</button>

    <button class="stock-btn" data-symbol="GOOGL">GOOGL</button>

    <button class="stock-btn" data-symbol="TSLA">TSLA</button>

    <button class="stock-btn" data-symbol="AMZN">AMZN</button>

  </div>

</section>
```

6.2 : script.js – Stock Search and API Integration

```
const API_KEY = "YOUR_API_KEY";

const stockInput = document.getElementById("stockInput");

const searchBtn = document.getElementById("searchBtn");
searchBtn.addEventListener("click", () => {
  searchStock(stockInput.value);
});
stockInput.addEventListener("keypress", (event) => {
  if (event.key === "Enter") {
    searchStock(stockInput.value);
  }
});
async function searchStock(symbol) {
  symbol = symbol.trim().toUpperCase();
  if (!symbol) {
    showError("Please enter a stock symbol.");
    return;
  }
  try {
    showLoading();
    const url =
      `https://www.alphavantage.co/query?function=TIME_SERIES_DAILY` +
      `&symbol=${symbol}&apikey=${API_KEY}`;
    const response = await fetch(url);
    const data = await response.json();
    if (data.Note || data["Error Message"]) {
      throw new Error("Unable to retrieve stock data.");
    }
    processStockData(symbol, data);
  } catch (error) {
    showError(error.message);
  }
```

```
}
```

6.3 : App.jsx (React Router – client-side routing)

```
 body {
   margin: 0;
   font-family: Arial, sans-serif;
   background: #080b18;
   color: #ffffff;
}

.container {
   width: 90%;
   max-width: 1250px;
   margin: auto;
}

.stock-analysis,
.chart-section,
.watchlist-section {
   background: rgba(255, 255, 255, 0.04);
   border: 1px solid rgba(0, 212, 255, 0.15);
   border-radius: 16px;
   padding: 25px;
   margin-bottom: 25px;
   box-shadow: 0 10px 30px rgba(0, 0, 0, 0.25);
}

.stat-card {
   padding: 20px;
   border-radius: 12px;
   background: rgba(255, 255, 255, 0.03);
   transition: transform 0.3s ease;
}

.stat-card:hover {
   transform: translateY(-4px);
}

.stock-stats {
   display: grid;
   grid-template-columns: repeat(5, 1fr);
   gap: 15px;
}
```

7. TESTING AND DEBUGGING

Introduction

Testing and debugging are important stages in the development of the Stock Market Analyzer. Testing helps verify that each feature performs according to its intended functionality, while debugging helps identify and resolve errors encountered during development.

During development, different functionalities were tested individually and together. These included stock searching, API communication, data processing, price calculations, chart generation, watchlist operations, and responsive behavior.

Types of Testing

1. Functional Testing

Functional testing was performed to verify that the major features of the application work correctly.

For example, the stock search feature was tested using valid stock symbols to verify that the corresponding market data is retrieved and displayed correctly.

2. Input Validation Testing

The search field was tested with empty input and improperly entered values to verify that the application handles invalid input appropriately.

3. API Testing

The API integration was tested to verify that the application can retrieve the expected daily stock data and properly handle API errors or unavailable responses.

4. Chart Testing

The chart functionality was tested to ensure that historical closing prices are correctly displayed and that a previous chart is removed before a new chart is generated.

5. Watchlist Testing

The watchlist was tested by adding, removing, and selecting stock symbols. LocalStorage behavior was also tested to ensure that saved stocks remain available within the browser.

6. Responsive Testing

The application was tested at different screen sizes to verify that the interface remains usable on desktop, tablet, and mobile-sized displays.

TEST CASES:-

Test ID	Description	Input	Expected Result	Status
TC-1	Search valid stock	AAPL	Stock information displayed	Pass
TC-2	Search another valid stock	MSFT	Correct stock information displayed	Pass
TC-3	Search with empty input	Empty	Validation/error message displayed	Pass
TC-4	Press Enter to search	Valid symbol + Enter	Stock search executed	Pass
TC-5	Display price information	Valid stock	Price and statistics displayed	Pass
TC-6	Display historical chart	Valid stock	30-day closing chart displayed	Pass
TC-7	Add stock to watchlist	AAPL	Stock added to watchlist	Pass
TC-8	Remove stock from watchlist	AAPL	Stock removed	Pass
TC-9	Select watchlist stock	Saved symbol	Stock searched again	Pass
TC-10	Responsive layout	Mobile screen	Layout adapts correctly	Pass

8. CONCLUSION AND FUTURE ENHANCEMENT

The **Stock Market Analyzer** is a web-based application developed to provide users with a simple and interactive way to search and analyze stock market information.

The system successfully combines HTML5, CSS3, JavaScript, Fetch API, Alpha Vantage API, Chart.js, and LocalStorage to create an interactive financial dashboard.

The application allows users to search for stock symbols and view important information such as the latest available closing price, price change, percentage change, opening price, highest price, lowest price, previous closing price, and trading volume.

The historical chart provides a graphical representation of closing prices from the latest 30 available trading days, making it easier for users to observe recent stock price movements.

The watchlist functionality provides additional convenience by allowing users to save frequently viewed stocks using browser LocalStorage.

The project also provided practical experience in API integration, asynchronous JavaScript, JSON data processing, DOM manipulation, data visualization, browser storage, responsive web design, and user interface development.

Overall, the Stock Market Analyzer demonstrates how modern frontend web technologies can be combined with an external financial data API to build a useful and interactive stock information application.

Future Enhancement

Although the current Stock Market Analyzer provides the essential functionality for searching and viewing stock information, several improvements can be added in the future.

1. Secure API Integration

The API communication can be moved to a backend or serverless function so that the API key is not exposed directly in client-side JavaScript.

2. Real-Time Market Data

A future version can integrate a suitable real-time or streaming market-data service to provide live intraday prices instead of relying only on daily time-series information.

3. Advanced Technical Analysis

Additional technical indicators such as moving averages, RSI, MACD, Bollinger Bands, and other analytical tools can be introduced.

4. User Authentication

User accounts can be implemented so that watchlists can be stored securely and synchronized across multiple devices.

5. Portfolio Management

A portfolio module can be added where users can enter their holdings and track estimated portfolio performance.

6. Price Alerts

The system can provide notifications when a selected stock reaches a user-defined target price.

7. More Historical Data

The application can provide longer historical periods and allow users to switch between different time ranges.

8. Advanced Stock Comparison

Users could compare multiple stocks using charts and performance indicators.

9. News Integration

A financial news API could be integrated to display relevant market news related to the selected company.

10. Progressive Web Application

The application could be converted into a Progressive Web App so that users can install and access it more conveniently on mobile devices.

With these future improvements, the Stock Market Analyzer can be developed into a more comprehensive financial analysis and portfolio-monitoring platform.

9. REFERENCES

Alpha Vantage – Official Documentation
For stock market API integration and financial time-series data.
Website: https://www.alphavantage.co/documentation/

Chart.js – Official Documentation
For creating responsive charts and graphical data visualization.
Website: https://www.chartjs.org/docs/latest/

MDN Web Docs
For HTML, CSS, JavaScript, Fetch API, DOM manipulation, and Web APIs.
Website: https://developer.mozilla.org/

W3Schools
For HTML, CSS, JavaScript, and frontend web development references.
Website: https://www.w3schools.com/

JavaScript Documentation
For JavaScript functions, arrays, objects, asynchronous operations, event handling, and browser storage.
Website: https://developer.mozilla.org/en-US/docs/Web/JavaScript

Brainware University Class Notes and Study Materials
For concepts and guidance related to web development and project implementation.
Website: https://www.brainwareuniversity.ac.in/